



Trinity College Dublin

Coláiste na Tríonóide, Baile Átha Cliath

The University of Dublin

SCHOOL OF COMPUTER SCIENCE AND STATISTICS

CONTINUE DEVELOPMENT OF (PARTS OF) AN INTERNET SCANNING INFRASTRUCTURE TAILORED TO IRELAND

SRIRAM KIRTHIVAS

PROF. STEPHEN FARRELL

JUNE 2, 2026

SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE DEGREE OF
M.SC. COMPUTER SCIENCE - FUTURE NETWORKED SYSTEMS

Declaration

I hereby declare that this Dissertation is entirely my own work and that it has not been submitted as an exercise for a degree at this or any other university.

I have read and I understand the plagiarism provisions in the General Regulations of the University Calendar for the current year, found at <http://www.tcd.ie/calendar>.

I have also completed the Online Tutorial on avoiding plagiarism 'Ready Steady Write', located at <http://tcd-ie.libguides.com/plagiarism/ready-steady-write>.

Signed: _____

Date: _____

Abstract

Acknowledgements

I would like to thank Prof. Stephen Farrell for his supervision and guidance throughout this project, and for providing access to the surveys scanning infrastructure that this work builds upon.

Contents

Abstract	ii
1 Introduction	1
1.1 Motivation	1
1.2 Objectives	2
1.2.1 Project Objectives	2
1.2.2 Personal Objectives	3
1.3 Structure of the Thesis	3
2 Background	4
2.1 Email Protocols and Architecture	4
2.2 TLS and Certificate Infrastructure	5
2.2.1 Let's Encrypt and ACME	7
2.2.2 Cipher Suites	8
2.3 SSH and Host Keys	9
2.4 STARTTLS vs Implicit TLS	10
2.5 Key Algorithms	11
2.6 Internet-Wide Scanning	11
2.6.1 ZMap	11
2.6.2 ZGrab and FreshGrab	12
2.6.3 SameKeys	12
2.7 Previous Work	13

3	Experimental Methods	14
4	Results and Discussion	15
5	Evaluation	16
6	Conclusions and Future Work	17
A1	Appendix	20
	A1.1 Appendix numbering	20

List of Figures

2.1	End-to-end email delivery architecture. The sender's mail client submits the message via SMTP on port 587, the sender's server relays it to the recipient's server via SMTP on port 25, and the recipient retrieves it using POP3 or IMAP. Port 25 is the internet-facing port scanned in this research.	5
2.2	TLS 1.2 handshake sequence between a client and a mail server. The client and server exchange supported parameters, then the server sends its certificate containing the public key (step 3, highlighted in yellow). This certificate is the cryptographic material that this dissertation collects and analyses for key reuse across Irish mail servers.	6
2.3	Automated certificate renewal using Let's Encrypt and the ACME protocol. Certbot on the mail server requests a new certificate from Let's Encrypt every 90 days, receiving a fresh certificate and private key each time. This automated cycle reduces key reuse because each renewal generates a unique key pair.	8
2.4	SSH host key reuse across two servers. When Server A and Server B present the same host key, compromising the private key of one server allows an attacker to impersonate the other. This commonly occurs when servers are cloned from a shared template image without regenerating keys.	9

2.5	Comparison of STARTTLS and Implicit TLS connection methods. STARTTLS begins with a plaintext connection on standard ports before upgrading to encryption mid-session, while Implicit TLS uses dedicated ports where encryption starts immediately with no plaintext phase. Both methods ultimately result in a TLS handshake that presents the server's certificate.	10
2.6	The scanning pipeline used in this dissertation. Irish IPv4 addresses are identified using the MaxMind GeoLite2 database, ZMap performs a SYN scan on port 25 to find open hosts, FreshGrab connects to those hosts to collect TLS certificates and SSH keys across multiple ports, and SameKeys groups hosts that share at least one cryptographic key into reuse clusters.	11

List of Tables

Nomenclature

1 | Introduction

This chapter provides an overview of the project, including its motivation, objectives, and structure. It sets the context for the research on cryptographic key reuse and outlines the significance of the study in understanding security practices on the Internet.

1.1 Motivation

Cryptographic keys are what keep internet connections private. Every SSH login and every TLS-secured email depends on the assumption that each server has its own unique key. But in practice, keys get reused across multiple servers, whether through cloned VM images, shared hosting templates, or simple misconfiguration. If an attacker compromises one server's private key, they can impersonate every other server that shares it.

Prof. Farrell's earlier work (1) showed this is not a theoretical concern. A 2018 scan of Irish mail servers found 1,437 clusters of hosts sharing keys, with the largest cluster containing nearly 2,000 hosts. Since then, cloud adoption has grown, Let's Encrypt has become the dominant certificate authority, and hosting practices have changed considerably. The question is whether key reuse has actually gotten better or whether it has just shifted into different patterns.

That is what motivated this project. I wanted to re-run the same kind of scan, using the same core methodology but with updated tools and data, and see what the Irish

internet looks like now. Repeating a measurement study like this is valuable because single snapshots only tell you so much. Comparing results across years is what lets you spot trends and say whether things are actually improving.

1.2 Objectives

This project has both project objectives and personal objectives. The project objectives focus on the research and technical aspects of investigating cryptographic key reuse, while the personal objectives aim to enhance my skills and knowledge in relevant areas.

1.2.1 Project Objectives

The primary objective of this project is to investigate how common cryptographic key reuse is among Irish mail servers in 2026. Based on the previous research done on this project, the project aims to:

- Understand Prof. Farrell's previous work (1) on key reuse and the existing scanning infrastructure.
- Update the MaxMind GeoLite2 IP geolocation database to get current Irish IP ranges.
- Set up a VPS with outbound port 25 access and configure research identification so network administrators can contact me if needed.
- Re-run the full scanning pipeline for Ireland and compare results against previous scans.
- Improve data visualisation of the results.
- Contribute improvements back to the upstream repository via a pull request.

1.2.2 Personal Objectives

The personal objectives of this project are to:

- Gain a deeper understanding of cryptographic protocols and their security implications.
- Gain hands-on experience with network scanning tools like ZMap and ZGrab.
- Develop skills in large-scale data collection and analysis.
- Enhance proficiency in shell scripting and Python for data processing.
- Get more comfortable with Git workflows, especially working with forks and pull requests.

1.3 Structure of the Thesis

This thesis is structured as follows:

- **Chapter 2 - Background:** Provides background information on the protocols involved, including email architecture, TLS, SSH, STARTTLS, certificate infrastructure, and the scanning tools used.
- **Chapter 3 - Experimental Methods:** Describes the implementation details of the project, including the methodology used for data collection and analysis.
- **Chapter 4 - Results and Discussion:** Presents the results of the scan, including cluster statistics and visualisations, with comparisons to previous years.
- **Chapter 5 - Evaluation:** Evaluates the findings and discusses their significance.
- **Chapter 6 - Conclusions and Future Work:** Summarises the key findings and suggests possible directions for future work.

2 | Background

This chapter includes the technical background that is necessary for understanding the remainder of this dissertation. It begins with an explanation of how email functions at the protocol level. Following that, it addresses how TLS and SSH provide protection for those connections. Lastly, it examines the various tools and techniques that are utilized for this scanning process.

2.1 Email Protocols and Architecture

Email functions based on several protocols that have existed for many years. The primary protocol is SMTP which stands for Simple Mail Transfer Protocol (2). This protocol has the job of sending mail between servers. When a person sends an email, their mail client connects with a local mail server through SMTP. After that, the local mail server forwards the email to the mail server of the recipient using SMTP again. The recipient retrieves the email using either POP3 or IMAP, depending on the email setup they have.

SMTP uses port 25 for communication between servers. Port 587 is set aside for mail submission, which occurs when a user's mail client sends an email to their local mail server. This difference is important because port 25 is the port that can be accessed on the internet and it is the port that is being looked at in this research.

POP3 which stands for Post Office Protocol version 3 operates on port 110 for unencrypted communication and port 995 for the TLS encrypted variant. POP3

downloads emails onto the client and removes them from the server. IMAP which stands for Internet Message Access Protocol operates on port 143 for unencrypted communication and port 993 for TLS. IMAP retains emails on the server and permits the client to synchronize across various devices.

All of these protocols can be secured using TLS to safeguard the connection. When TLS is employed, the server presents a certificate that contains a public key. This public key is what we gather and examine for reuse among different servers. Figure 2.1 illustrates this end-to-end flow.

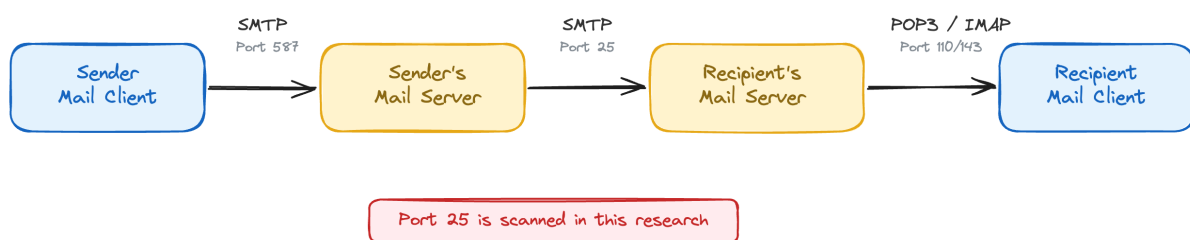


Figure 2.1: End-to-end email delivery architecture. The sender's mail client submits the message via SMTP on port 587, the sender's server relays it to the recipient's server via SMTP on port 25, and the recipient retrieves it using POP3 or IMAP. Port 25 is the internet-facing port scanned in this research.

2.2 TLS and Certificate Infrastructure

Transport Layer Security (TLS) (3) encrypts connections between clients and servers, replacing the older SSL protocol as the standard for securing web traffic, email, and other internet services. When a client connects using TLS, both sides perform a handshake to establish an encrypted session. Figure 2.2 shows this process for TLS 1.2.

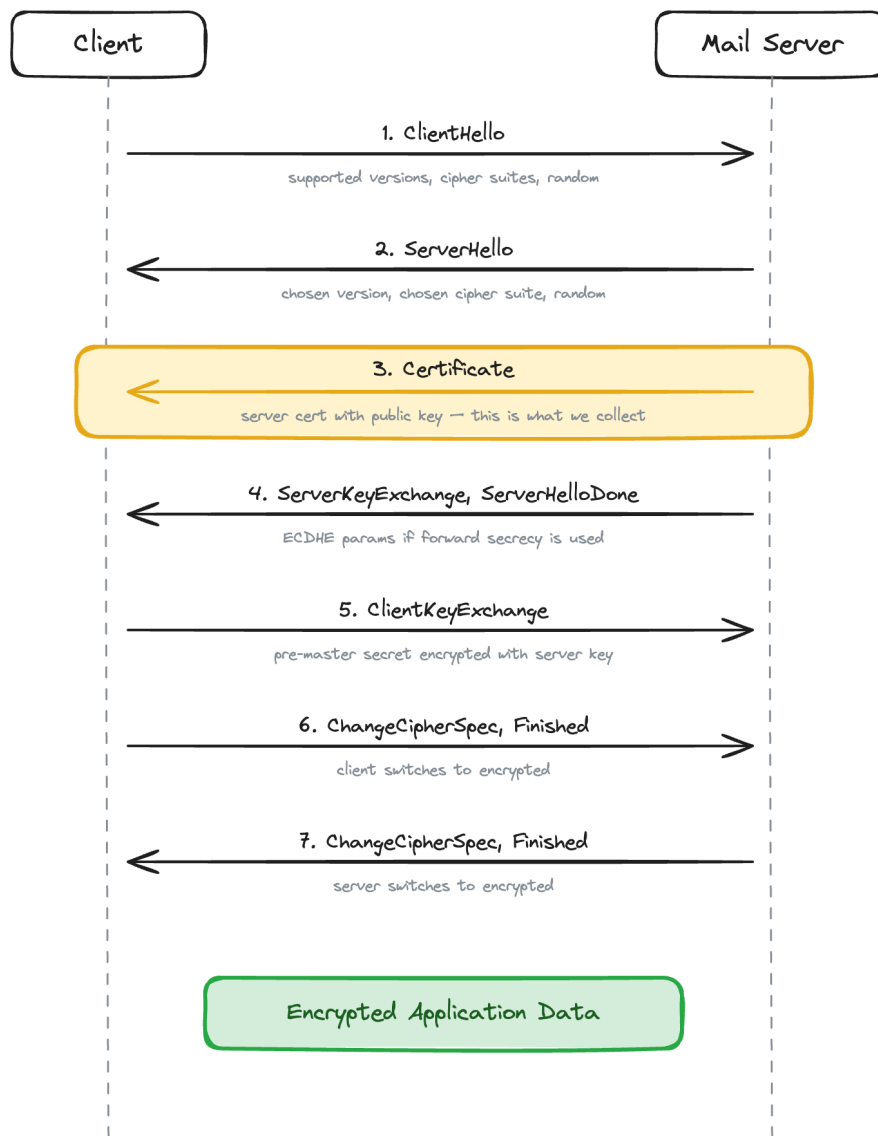


Figure 2.2: TLS 1.2 handshake sequence between a client and a mail server. The client and server exchange supported parameters, then the server sends its certificate containing the public key (step 3, highlighted in yellow). This certificate is the cryptographic material that this dissertation collects and analyses for key reuse across Irish mail servers.

Certificates have a validity period with a start and end date. Expired certificates should be rejected by clients, but many mail servers continue using them because SMTP does not enforce certificate validation the way HTTPS does. Certificates can also be self-signed, providing encryption without authentication.

The public key within the certificate is the most important element for this dissertation. If two servers present certificates with the same public key, either the

same operator is reusing one key across servers or key generation has failed. In both cases, compromise of the private key would allow an attacker to impersonate or decrypt traffic for all servers sharing that key.

2.2.1 Let's Encrypt and ACME

Let's Encrypt (4) is a free Certificate Authority that started in the year 2015 and has had a large effect on the adoption of TLS. Before Let's Encrypt was available, getting a certificate meant that one had to pay a commercial Certificate Authority which caused many small server operators to opt for self-signed certificates rather than spend money on commercial certificates.

Let's Encrypt offers certificates that remain valid for 90 days and uses the ACME protocol to make the process of issuing and renewing certificates fully automated. A server operator can install a tool such as Certbot (5), execute it once, and thereafter the certificates will renew automatically. The 90-day renewal cycle is significant because it ensures that Let's Encrypt servers obtain fresh certificates and keys every 90 days, compared to commercial Certificate Authorities that issue certificates valid for one year or longer.

In this research, the adoption of Let's Encrypt is significant because it has a direct impact on patterns of key reuse. When a hosting provider utilizes Let's Encrypt with automated renewal, each server receives its own distinct certificate and key, which leads to a decrease in the type of key sharing that is being investigated. The increase in the use of Let's Encrypt is one of the factors contributing to the reduction in the number of key reuse clusters when compared to previous studies. Figure 2.3 shows the automated renewal flow.

2.3 SSH and Host Keys

SSH is the Secure Shell protocol (9) that is utilized for remote authentication of servers and it operates on port 22 while using host keys for server identification.

When a client makes a connection to an SSH server for the first time, it is given the server's host key and this key is stored by the client. For any future connections, the client will verify that the key has not changed and if there is a change in the key, the client will provide a warning to the user indicating that the server identity has changed.

SSH host keys serve the same function as TLS certificates since they identify a server through a public key. In situations where two servers have the same SSH host key, if an attacker is able to compromise one of the servers, that attacker can then impersonate the other server. The reuse of SSH host keys can occur when servers are duplicated from a template image without creating new keys or when a hosting provider utilizes the same base image across all of their customers. Figure 2.4 illustrates this risk.

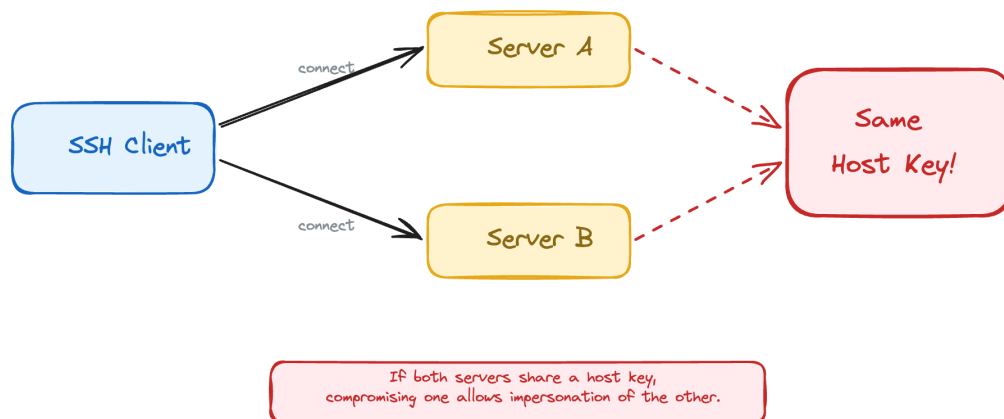


Figure 2.4: SSH host key reuse across two servers. When Server A and Server B present the same host key, compromising the private key of one server allows an attacker to impersonate the other. This commonly occurs when servers are cloned from a shared template image without regenerating keys.

2.4 STARTTLS vs Implicit TLS

There are two ways TLS is used with mail protocols. STARTTLS is a protocol extension where the client connects on the plaintext port (25, 110, 143) and then upgrades the connection to TLS by issuing a STARTTLS command. The connection starts unencrypted and switches to encrypted mid-session. Implicit TLS uses a separate port (465, 993, 995) where the connection is encrypted from the first byte. No plaintext phase exists.

In this research, ZGrab attempts STARTTLS on ports 25, 110, 143, and 587, and connects with implicit TLS on port 993. Both methods result in a TLS handshake where the server presents its certificate, which is what we collect. Figure 2.5 compares the two approaches.

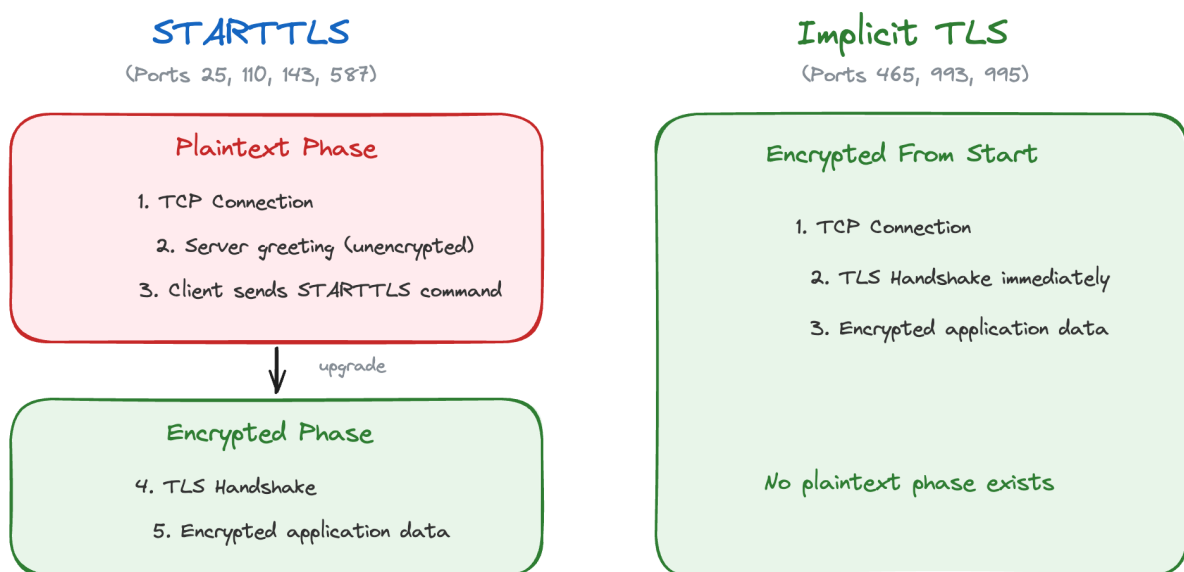


Figure 2.5: Comparison of STARTTLS and Implicit TLS connection methods. STARTTLS begins with a plaintext connection on standard ports before upgrading to encryption mid-session, while Implicit TLS uses dedicated ports where encryption starts immediately with no plaintext phase. Both methods ultimately result in a TLS handshake that presents the server's certificate.

2.5 Key Algorithms

TLS certificates contain a public key that uses one of several cryptographic algorithms. RSA is the most common, and keys are identified by their bit length. RSA-2048 is the current minimum recommended key size. RSA keys below 2048 bits (such as RSA-1024) are considered weak because they can be factored with sufficient computational resources. RSA-4096 provides a larger security margin.

ECDSA (Elliptic Curve Digital Signature Algorithm) is an alternative to RSA that provides equivalent security with shorter key lengths. A 256-bit ECDSA key provides comparable security to a 3072-bit RSA key. ECDSA adoption is growing, particularly through managed services like Amazon SES.

2.6 Internet-Wide Scanning

Scanning the entire internet, or a subset of it like all Irish IP addresses, is a technique that has become practical in the last decade thanks to tools designed for it. Figure 2.6 shows the complete scanning pipeline used in this dissertation.

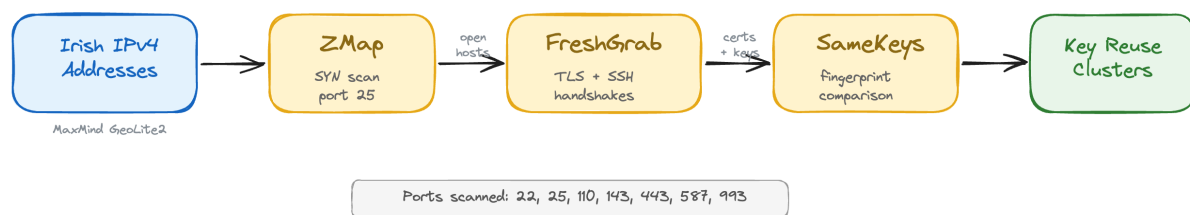


Figure 2.6: The scanning pipeline used in this dissertation. Irish IPv4 addresses are identified using the MaxMind GeoLite2 database, ZMap performs a SYN scan on port 25 to find open hosts, FreshGrab connects to those hosts to collect TLS certificates and SSH keys across multiple ports, and SameKeys groups hosts that share at least one cryptographic key into reuse clusters.

2.6.1 ZMap

ZMap (10) is a stateless network scanner developed by Durumeric et al. at the University of Michigan. It can scan the entire IPv4 address space on a single port in

under an hour. ZMap works by sending one SYN packet to each target IP and recording which ones respond with a SYN-ACK, indicating that the port is open.

The key design decision in ZMap is that it is stateless. Traditional scanners like Nmap maintain a table of connections and wait for responses, which limits how fast they can go. ZMap does not keep track of outstanding probes. It encodes the target information into the packet itself and uses a technique based on cyclic multiplicative groups to visit IP addresses in a random order without repeating any.

For this research, ZMap was used to scan all Irish IPv4 addresses on port 25 to find mail servers. The Irish address space was determined using the MaxMind GeoLite2 database (11), which maps IP prefixes to countries.

2.6.2 ZGrab and FreshGrab

Once ZMap identifies which hosts have a port open, the next step is to connect to those hosts and collect information. ZGrab performs application-layer handshakes, such as TLS handshakes on mail ports or SSH key exchanges on port 22, and records the responses.

FreshGrab is a wrapper around ZGrab that was developed as part of the surveys project this dissertation builds on. It takes the list of IPs found by ZMap and connects to each one across multiple ports, collecting TLS certificates and SSH host keys from ports 22, 25, 110, 143, 443, 587, and 993. The output is a JSON file with one line per IP containing all the cryptographic material collected.

2.6.3 SameKeys

SameKeys is a Python script from the surveys project that takes the output of FreshGrab and looks for cryptographic key reuse. It compares the public key fingerprints across all hosts and groups them into clusters where two or more hosts share at least one key.

The output is a JSON file listing all the clusters with their member IPs, the ports on which the shared keys were found, and the key types involved. In the 2018 study (1), SameKeys found 1,437 clusters across Irish mail servers. This dissertation reruns the same tool on 2026 data and compares the results.

2.7 Previous Work

The most directly relevant previous work is the 2018 study by Farrell (1), which performed the same kind of scanning on Irish mail servers and found significant key reuse. That study used ZMap to find hosts, ZGrab to collect keys, and SameKeys to identify clusters. It found 1,437 clusters of hosts sharing cryptographic keys, with 53% of all scanned hosts belonging to at least one cluster.

Durumeric et al. (12) conducted a large scale analysis of the HTTPS certificate ecosystem in 2013, which demonstrated that internet-wide scanning could reveal widespread problems in how certificates are managed. Their work showed that many certificates were shared across multiple hosts, because of hosting providers deploying the same certificate to all their customers.

Heninger et al. (13) in 2012 showed that a significant number of TLS and SSH keys on the internet shared common factors, meaning their random number generation was flawed. This was a different kind of key reuse where the keys were not identical but were mathematically related in a way that allowed the private key to be recovered.

3 | Experimental Methods

4 | Results and Discussion

5 | Evaluation

6 | Conclusions and Future Work

Bibliography

- [1] Stephen Farrell. Clusters of re-used keys. Cryptology ePrint Archive, Paper 2018/299, 2018. URL <https://eprint.iacr.org/2018/299>.
- [2] John Klensin. RFC 5321: Simple Mail Transfer Protocol (SMTP). <https://datatracker.ietf.org/doc/html/rfc5321>, 2008.
- [3] Tim Dierks and Eric Rescorla. RFC 5246: The Transport Layer Security (TLS) Protocol Version 1.2, 2008.
- [4] Internet Security Research Group. Let’s encrypt. <https://letsencrypt.org>, 2015.
- [5] Electronic Frontier Foundation. Certbot. <https://certbot.eff.org>, 2024.
- [6] Andrey Popov. RFC 7465: Prohibiting RC4 Cipher Suites, 2015.
- [7] Thai Duong and Juliano Rizzo. Beast – here come the xor ninjas, 2011.
- [8] Nadhem J Al Fardan and Kenneth G Paterson. Lucky thirteen: Breaking the tls and dtls record protocols. In *2013 IEEE symposium on security and privacy*, pages 526–540. IEEE, 2013.
- [9] Tatu Ylonen and Chris Lonvick. RFC 4253: The Secure Shell (SSH) Transport Layer Protocol. <https://datatracker.ietf.org/doc/html/rfc4253>, 2006.
- [10] Zakir Durumeric, Eric Wustrow, and J. Alex Halderman. Zmap: Fast internet-wide scanning and its security applications. In *22nd USENIX Security Symposium*, 2013.

- [11] MaxMind. Maxmind geolite2 databases. <https://www.maxmind.com>, 2024.
- [12] Zakir Durumeric, James Kasten, Michael Bailey, and J Alex Halderman.
Analysis of the https certificate ecosystem. In *Proceedings of the 2013 conference on Internet measurement conference*, pages 291–304, 2013.
- [13] Nadia Heninger, Zakir Durumeric, Eric Wustrow, and J Alex Halderman.
Mining your ps and qs: Detection of widespread weak keys in network devices.
In *21st USENIX Security Symposium (USENIX Security 12)*, pages 205–220, 2012.

A1 | Appendix

You may use appendices to include relevant background information, such as calibration certificates, derivations of key equations or presentation of a particular data reduction method. You should not use the appendices to dump large amounts of additional results or data which are not properly discussed. If these results are really relevant, then they should appear in the main body of the report.

A1.1 Appendix numbering

Appendices are numbered sequentially, A1, A2, A3... The sections, figures and tables within appendices are numbered in the same way as in the main text. For example, the first figure in Appendix A1 would be Figure A1.1. Equations continue the numbering from the main text.